

Homework 3: HL7 Parsing and Modification

Jonathan DeLemos (PennKey: delemos1)

Summary of Approach

The instructions for this assignment were clear and well-scoped. After reading them through, I completed the code in the order the template laid out: read `sample-hl7.txt`, normalize line endings, parse with the `hl7` Python library, extract the requested fields into a dictionary, dump it to `output.json`, then mutate the in-memory message to apply the required changes and write the result to `modified-hl7.txt`. I used `hl7inspector.com` to confirm the field indices for each segment (PID, PV1, OBR, DG1, OBX) before writing the access code.

HL7 vs. Other Text-Based Data, and vs. DICOM

HL7 looks like text but behaves more like a delimited, positionally-encoded record format. Unlike JSON, CSV, or XML, there are no field names in the payload — every field is identified by its ordinal position within a pipe-delimited segment, and the meaning of each position is defined externally by the HL7 standard and message version. There is also a five-level nesting hierarchy (segment → field → repetition → component → sub-component) using several different delimiters (`\r`, `|`, `~`, `^`, `&`), and segments terminate on carriage returns rather than newlines, which is unusual on modern systems. DICOM, by contrast, is a binary, tag-based format: each element is identified by a numeric (`group`, `element`) tag with an explicit value representation describing its type, length, and encoding, and the payload can carry pixel data alongside metadata. Where HL7 is a structured text telegram designed for messaging between information systems, DICOM is a self-describing binary container designed for imaging and persistence — so DICOM effectively requires a library to read, while HL7 *could* be parsed with string operations (though doing so is unwise given the delimiter hierarchy and escape rules).

Accommodations in My Code

Two accommodations were needed. First, the source file used lone carriage returns as segment terminators, so the parser would not split segments correctly until I explicitly replaced `\n` with `\r` in the loaded string (Python's universal-newlines mode converts `\r` to `\n` on read). Second, several fields are positional but only meaningful in a deeper component — e.g., OBR-31 stores the reason for study in the *second* component of the field (the first is empty), so I had to drill in with `obr[31][0][1]` rather than just `obr[31]`. For the patient name I extracted the family and given names from PID-5 as separate components and assembled them into a human-readable “First Last” string, while leaving the referring doctor field as-is per the assignment instructions.

Highest Source of Errors

The biggest hindrance was unfamiliarity with the `hl7` Python package — specifically, getting comfortable with the shape of the outputs returned by its accessors. The library returns nested container types (`Message`, `Segment`, `Field`, `Repetition`, `Component`) rather than plain strings, so my early attempts produced values that looked correct when printed but were actually wrapped objects with subtly wrong sub-indexing. For example, my first attempt at the reason-for-study extraction was:

```
reason_for_study = obr[31] # returns a Field, not a string
```

which serialized as `^Abd pain and weight loss, non-acute` — including the leading caret because the first component was empty. The text actually lives in the second component of the first repetition, so the correct access is:

```
reason_for_study = str(obr[31][0][1]) # 'Abd pain and weight loss, non-acute'
```

I resolved this class of bug by writing a small exploration script that printed `repr()` of each field I needed, which revealed the exact nesting depth (e.g. PID[5] surfaced as `[[['TIGER'], ['DANIELLE']]]`, telling me I needed two index steps to reach the family name and to wrap the result in `str()` for clean JSON output). Once I had that mental model of the package's return types, the rest of the extractions and in-place modifications fell out cleanly.